analytical_exercises/rnn_6f.py

```python
#!/usr/bin/env python3
"""
Problem 6(f) – Bonus-2
RNN trainer for a fixed sequence 101010, using BPTT with a loss at every time step.

RNN model (same recurrence as before):
    s_k = w_rec * s_{k-1} + w_x * x_k

But now:
    • At each time step k, the "desired" output is the cumulative number of ones so far:
          u_k = sum_{i=1}^k x_i
    • We interpret the hidden state as the output at each step, y_k = s_k.
    • The total loss is the average over time:
          L = (1/T) * sum_{k=1}^T (u_k - s_k)^2

We implement BPTT with this loss (see 6(e) and the D2L link).
"""

import os
import random
from typing import List, Tuple


# ---------------------------------------------------------------
# Forward pass: compute all states s_0,...,s_T
# ---------------------------------------------------------------

def forward_rnn(seq: List[int], w_rec: float, w_x: float) -> List[float]:
    """
    Forward pass for a single sequence.

    Args:
        seq:   list of bits (0/1), e.g. [1, 0, 1, 0, 1, 0]
        w_rec: recurrent weight
        w_x:   input weight

    Returns:
        states: list [s_0, s_1, ..., s_T] for use in backprop
    """
    s = 0.0          # initial state s_0
    states = [s]     # store s_0

    for x in seq:
        # recurrence: s_k = w_rec * s_{k-1} + w_x * x_k
        s = w_rec * s + w_x * x
        states.append(s)

    return states


# ---------------------------------------------------------------
```

```python
# Loss + Backward pass (BPTT with per-time-step loss)
# ----------------------------------------------------------------

def loss_and_backward_rnn(
    seq: List[int],
    states: List[float],
    w_rec: float
) -> Tuple[float, float, float]:
    """
    Compute the average loss over all time steps and the gradients using BPTT.

    Model and loss:
        s_k = w_rec * s_{k-1} + w_x * x_k
        u_k = cumulative count of ones up to time k
        L   = (1/T) * sum_{k=1}^T (u_k - s_k)^2

    Let:
        δ_k = dL/ds_k

    Then (scalar version of the D2L formulas):
        local_k = dL_k/ds_k = (2/T) * (s_k - u_k)
        δ_k     = local_k + w_rec * δ_{k+1}
    and
        ∂L/∂w_x   = Σ_k δ_k * x_k
        ∂L/∂w_rec = Σ_k δ_k * s_{k-1}

    Args:
        seq:    input sequence (list of 0/1), length T
        states: list [s_0, ..., s_T] from forward pass
        w_rec:  recurrent weight used in forward

    Returns:
        loss:        average MSE over all time steps
        grad_w_rec:  dL/dw_rec
        grad_w_x:    dL/dw_x
    """
    T = len(seq)
    # Extract s_1,...,s_T; states[0] = s_0
    s_list = states[1:]

    # Compute cumulative targets u_k and loss
    cumulative = 0
    targets = []   # u_k for k=1..T
    loss = 0.0

    for k in range(T):
        cumulative += seq[k]          # u_k = number of ones up to step k+1
        u_k = cumulative
        targets.append(u_k)

        err = s_list[k] - u_k         # s_k - u_k
        loss += err * err

    loss /= T                         # average over time
```

```
106
107     # Backward pass (BPTT)
108     grad_w_rec = 0.0
109     grad_w_x = 0.0
110     delta_next = 0.0                  # δ_{T+1} = 0 (no future contribution)
111
112     # Go backwards: k = T-1,...,0   (step index k corresponds to time k+1)
113     for k in reversed(range(T)):
114         s_k = s_list[k]               # s_{k+1} in time notation
115         s_prev = states[k]            # s_k in time notation
116         x_k = seq[k]                  # x_{k+1}
117         u_k = targets[k]              # target up to step k+1
118
119         # Local gradient of L_k wrt s_k: (2/T)*(s_k - u_k)
120         local_grad = (2.0 / T) * (s_k - u_k)
121
122         # Total δ_k = local term + recurrent term from future
123         delta = local_grad + w_rec * delta_next
124
125         # Accumulate parameter gradients
126         grad_w_x   += delta * x_k
127         grad_w_rec += delta * s_prev
128
129         # Pass gradient to previous step
130         delta_next = delta
131
132     return loss, grad_w_rec, grad_w_x
133
134
135 # ------------------------------------------------------------
136 # Train the RNN for a single random initialization (new model)
137 # ------------------------------------------------------------
138
139 def train_one_run(
140     seq: List[int],
141     epochs: int,
142     eta: float,
143     init_seed: int
144 ) -> Tuple[float, float, float, float, float, float]:
145     """
146     Train the RNN on a single fixed sequence for a given number of epochs,
147     using the new per-time-step loss and BPTT.
148
149     Args:
150         seq:       fixed input sequence
151         epochs:    number of epochs (here: 15)
152         eta:       learning rate
153         init_seed: seed for random initialization
154
155     Returns:
156         (w_rec_init, w_x_init, initial_loss,
157          w_rec_final, w_x_final, final_loss)
158     """
159     rng = random.Random(init_seed)
```

```
160
161     # Random initialization of weights in a given range
162     w_rec = rng.uniform(-0.5, 0.9)
163     w_x   = rng.uniform(-0.5, 0.9)
164
165     # Compute initial loss before any update (with new loss definition)
166     states0 = forward_rnn(seq, w_rec, w_x)
167     initial_loss, _, _ = loss_and_backward_rnn(seq, states0, w_rec)
168     w_rec_init, w_x_init = w_rec, w_x
169
170     # Training loop: same sequence, repeated for 'epochs' steps
171     for _ in range(epochs):
172         # Forward pass
173         states = forward_rnn(seq, w_rec, w_x)
174
175         # Loss and gradients via BPTT (per-time-step loss)
176         loss, grad_wr, grad_wx = loss_and_backward_rnn(seq, states, w_rec)
177
178         # Gradient descent update
179         w_rec -= eta * grad_wr
180         w_x   -= eta * grad_wx
181
182     # Final loss after training
183     states_final = forward_rnn(seq, w_rec, w_x)
184     final_loss, _, _ = loss_and_backward_rnn(seq, states_final, w_rec)
185
186     return w_rec_init, w_x_init, initial_loss, w_rec, w_x, final_loss
187
188
189 # ------------------------------------------------------------
190 # Main: fixed sequence 101010, 15 epochs, save aligned table
191 # ------------------------------------------------------------
192
193 def main() -> None:
194     # Fixed input sequence 101010
195     seq = [1, 0, 1, 0, 1, 0]
196
197     epochs = 15
198     eta = 0.01              # learning rate
199
200     # 3 different random initializations (as in your version)
201     init_seeds = [2, 16, 20]
202
203     # Ensure results directory exists
204     os.makedirs("results", exist_ok=True)
205     output_path = os.path.join("results", "rnn_6f_results.txt")
206
207     with open(output_path, "w", encoding="utf-8") as f:
208         # General info header
209         f.write("Problem 6(f) – RNN counting ones with BPTT (per-time-step loss)\n")
210         f.write(f"Input sequence: {seq}\n")
211         f.write(f"Epochs: {epochs}\n")
212         f.write(f"Learning rate: {eta}\n\n")
213
```

```
        # Table header (aligned columns)
        header = (
            f"{'Run':>3} | "
            f"{'Seed':>4} | "
            f"{'w_rec_init':>12} | "
            f"{'w_x_init':>10} | "
            f"{'initial_loss':>13} | "
            f"{'w_rec_final':>12} | "
            f"{'w_x_final':>10} | "
            f"{'final_loss':>11}\n"
        )
        f.write(header)
        f.write("-" * (len(header) - 1) + "\n")  # separator line

        # One row per run
        for run_id, seed in enumerate(init_seeds, start=1):
            (
                w_rec_init,
                w_x_init,
                initial_loss,
                w_rec_final,
                w_x_final,
                final_loss,
            ) = train_one_run(
                seq=seq,
                epochs=epochs,
                eta=eta,
                init_seed=seed,
            )

            row = (
                f"{run_id:>3} | "
                f"{seed:>4} | "
                f"{w_rec_init:12.6f} | "
                f"{w_x_init:10.6f} | "
                f"{initial_loss:13.6f} | "
                f"{w_rec_final:12.6f} | "
                f"{w_x_final:10.6f} | "
                f"{final_loss:11.6f}\n"
            )
            f.write(row)

    print(f"Results written to: {output_path}")


if __name__ == "__main__":
    main()

```